

Estudo de caso TransCarga

(implementando autenticação e autorização com JAVA EE)

O sistema TransCarga é uma aplicação web para gerenciar o transporte de cargas, organizada em torno de papéis de usuário (USER) e administrador (ADMIN), e incorpora fluxos de autenticação (usuário e senha), autorização RBAC (baseado em papéis) e operações CRUD básicas.

Objetivo principal do estudo de caso é entender toda a parte de autenticação e autorização do sistema usando o Java EE (atualmente JAKARTA EE)

Seguem os casos de uso definidos e regras de negócio para o sistema:

Login/Logout

- Login: qualquer usuário (tanto USER quanto ADMIN) acessa a página de login, informa usuário e senha, e, em caso de sucesso, recebe um objeto de sessão que guarda seu papel.
- Logout: encerra a sessão e redireciona ao login, garantindo que páginas protegidas não fiquem acessíveis.

Cadastro de Fretes

- Caso de Uso “Cadastrar Frete”: permitido apenas a ADMIN. Exibe um formulário HTML para inserir destino, peso e transportadora, e, ao submeter, o servlet grava o novo frete no banco.

Listagem de Fretes

- Caso de Uso “Listar Fretes”: disponível para ambos os papéis. Ao acessar a rota, o sistema consulta o banco via JPA/Hibernate e exibe uma tabela com todos os fretes cadastrados.

Cadastro de Usuários

- Caso de Uso “Cadastrar Usuário”: restrito a ADMIN. A interface HTML coleta usuário, senha (hashed com BCrypt) e papel (ADMIN ou USER), e o servlet persiste o novo registro na tabela de usuários.

Listagem de Usuários

- Caso de Uso “Listar Usuários”: apenas ADMIN pode visualizar todos os usuários cadastrados, em uma tabela semelhante à de fretes.

Regras de negócio na autenticação/autorização

Controle de Acesso (AuthFilter)

- Um filtro global (AuthFilter) intercepta todas as requisições e:
 - Permite páginas públicas (login, logout, cadastro de usuário).
 - Verifica se há um usuário na sessão; caso contrário, redireciona ao login.
 - Restringe ações conforme o papel: USER só acessa listagem de fretes, enquanto ADMIN acessa qualquer rota.

Em conjunto, esses casos de uso garantem um fluxo de trabalho simples e seguro: o usuário faz login, vê apenas o que lhe é permitido, e, conforme seu papel, executa ações de leitura ou também de criação de dados.

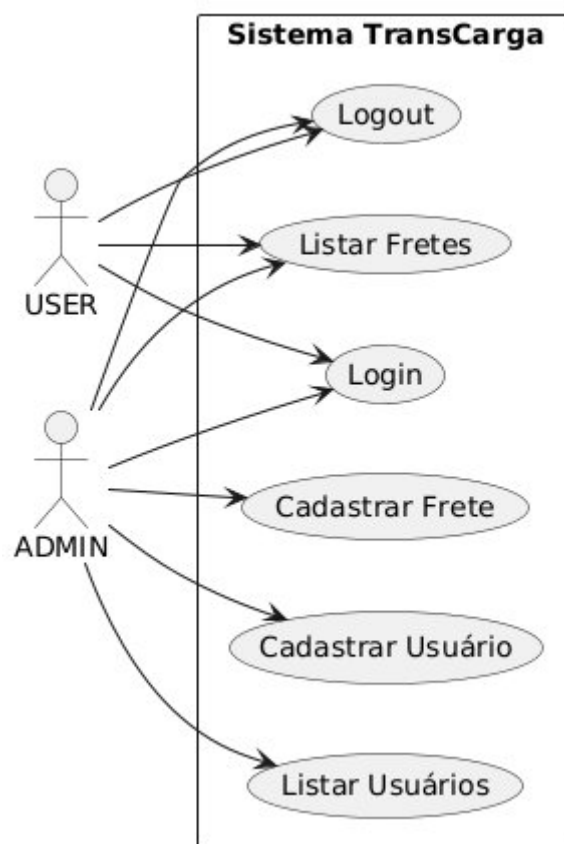


Diagrama de caso de uso



http://localhost:8080/transcarga/login.html

Login TransCarga

Usuário: admin

Senha:

Entrar

Tela de login



http://localhost:8080/transcarga/index.html

Bem-vindo ao sistema TransCarga

- [Cadastrar Frete](#)
- [Listar Fretes](#)
- [Cadastrar Usuário](#)
- [Listar Usuários](#)
- [Logout](#)

Tela Home do ADMIN



http://localhost:8080/transcarga/listarUsuarios.html

Lista de Usuários

login	Função
lima	ADMIN
mizael	ADMIN
aluno1	USER
user1	USER
user2	USER
adm1	ADMIN
user3	USER

Tela de listagem de contas (somente ADMIN)



The screenshot shows a web browser window with the address bar displaying 'http://localhost:8080/transcarga/listarFrete.html'. The page title is 'Lista de Fretes'. Below the title is a table with 4 columns: ID, Destino, Peso, and Transportadora. The table contains 9 rows of data.

ID	Destino	Peso	Transportadora
1	aasas	1212,00	1212
2	aa	1,00	2
3	aa	1,00	2
4	areia branca	100,00	TRDG
5	qq	12,00	DSA
6	GRossos	12,00	TRD
7	Tibau	30,00	THQ
8	Mossoró	12,00	12
9	Mossoró	32,00	TDHSA

Tela de Listar fretes (ADMIN e USER)

Vamos as explicações das funções principais de controle de autenticação e autorização com as classes **AuthFilter**, **LoginServlet** e **UsuarioServlet**

1- O **AuthFilter** é um filtro global que intercepta todas as requisições (@WebFilter("/*)) e primeiro libera as rotas públicas de login, logout e cadastro de usuário; em seguida verifica se existe um objeto User na sessão (autenticação) e, se não houver, redireciona ao login; depois, faz autorização por papel: se o usuário for do tipo USER só permite a rota de listar fretes (redirecionando qualquer outro pedido de volta a ela) enquanto o ADMIN recebe acesso irrestrito a todas as rotas; por fim, quando as condições de autenticação e autorização são atendidas, chama chain.doFilter() para continuar o fluxo normal da aplicação.

```
[...]
@WebFilter("/*")
public class AuthFilter implements Filter {

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain
chain) throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;
        String uri = req.getRequestURI();

        //Libera páginas públicas
        if (uri.endsWith("login") || uri.endsWith("login.html")) {
            chain.doFilter(request, response);
            return;
        }

        HttpSession session = req.getSession(false);
        User usuario = (session != null) ? (User) session.getAttribute("user") : null;

        if (usuario == null) {
            resp.sendRedirect("login.html");
            return;
        }
        // RESTRIÇÃO POR PAPEL
```

```

String role = usuario.getRole(); // "USER" ou "ADMIN"

if ("USER".equals(role)) {
    // Usuário comum só pode acessar a página de listar fretes
    if (uri.contains("listarFretes")) {
        chain.doFilter(request, response);
    } else {
        // Redireciona ou mostra erro de acesso
        resp.sendRedirect("listarFretes.html");
    }
} else if ("ADMIN".equals(role)) {
    // Admin tem acesso a tudo
    chain.doFilter(request, response);
} else {
    // Papel desconhecido
    resp.sendRedirect("login.html");
}
}
}

```

2- O **LoginServlet** processa requisições POST vindas do formulário de login, recebendo os parâmetros username e password, então utiliza o UserDAO para autenticar o usuário verificando se as credenciais conferem com o hash armazenado; se a autenticação for bem-sucedida, o servlet armazena o objeto User na sessão HTTP e redireciona para a rota protegida de listagem de fretes, caso contrário, redireciona de volta para a página de login com um indicador de erro, garantindo assim o estabelecimento de sessão apenas para usuários válidos.

```

[...]
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws IOException {
        String u = req.getParameter("username");
        String p = req.getParameter("password");
        User user = new UserDAO().autenticar(u, p);
        if (user != null) {
            req.getSession().setAttribute("user", user);
            resp.sendRedirect("index.html");
        } else {
            resp.sendRedirect("login.html?error=1");
        }
    }
}
[...]

```

3- O **UsuarioServlet** recebe requisições POST do formulário de cadastro de usuário (cadastrarUsuario.html), extrai os parâmetros username, password e role, e invoca o método cadastrar do UserDAO, que criptografa a senha com BCrypt e persiste o novo User no banco via JPA; em seguida, ele redireciona o usuário à página listarUsuarios.html, concluindo o fluxo de criação de contas de forma segura e delegando toda a lógica de armazenamento ao DAO.

```
@WebServlet(name = "usuario", urlPatterns = {"/usuario", "/listarUsuarios.html"})
public class UsuarioServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws IOException {
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        String role = request.getParameter("role");

        UserDAO dao = new UserDAO();
        dao.cadastrar(username, password, role);

        response.sendRedirect("listarUsuarios.html");
    }
}
```

Em resumo, em Java EE, **autenticação** é o processo de verificar a identidade de um usuário — tipicamente feito ao receber credenciais (como nome de usuário e senha) em um servlet ou via formulário, compará-las com dados armazenados (por exemplo, em um banco de dados usando JPA) e, se corretas, criar uma sessão HTTP onde o objeto `Principal` ou um atributo `user` indica quem está logado. Já a **autorização** ocorre depois da autenticação e define “o que” aquele usuário pode fazer: com base em seu **papel** (por exemplo, “USER” ou “ADMIN”), o sistema permite ou bloqueia o acesso a cada recurso (como servlets, páginas ou métodos de EJB).

Na prática, em Java EE você pode usar filtros (`javax.servlet.Filter`/`jakarta.servlet.Filter`) para interceptar requisições, verificar a sessão e o papel do usuário, e decidir se continua o fluxo (`chain.doFilter()`) ou redireciona para uma página de erro/login.

Acesso ao código fonte completo desse exemplo está disponível em <https://github.com/clistion/transCargaHTMLJava/tree/2-login-example-rbac>

FIM